Informed Search

David Izhaki
Department of Computer Science, Bar-Ilan

Today's Plan

- We'll introduce **informed search** and formally define what a **heuristic** is, plus why we need it.
- Algorithms we'll cover:
  1. **GBFS** (Greedy Best-First Search)
  2. **A*** (A-star)
  3. **IDA*** (Iterative Deepening A*)
  4. **Beam Search**

Outline

- **Part 1: Heuristics**
- Part 2: Informed Search Algorithms

Heuristic

- A **heuristic** is a quick rule-of-thumb that helps a search algorithm decide **which state to try next**.
- It gives an **estimate** of "how close" a state is to the goal.
- It is **not guaranteed to be correct**, the point is to be **fast** and **useful**, not perfect.
- Good heuristics reduce the number of states we explore, so search becomes much faster.

Why Do We Need Heuristics?

- State spaces explode
- Exact distance-to-goal is too expensive
- Heuristics give direction

Why Do We Need Heuristics?

- **State spaces explode**
  - Search trees grow extremely fast as depth increases.
  - Memory becomes a bottleneck because the frontier can become huge.
- Exact distance-to-goal is too expensive
- Heuristics give direction

Why Do We Need Heuristics?

- State spaces explode
- **Exact distance-to-goal is too expensive**
  - The true remaining cost is often computationally hard to compute. (PSPACE-hard)
  - So, although it may be computable in theory, the required time and memory can be prohibitive in practice.
- Heuristics give direction

Why Do We Need Heuristics?

- State spaces explode
- Exact distance-to-goal is too expensive
- **Heuristics give direction**
  - Heuristics focus the search on states that look more promising.
  - They help informed search algorithms explore fewer states overall.

Heuristic Formal Definitions

- **Heuristic function:**

$$h : S \to \mathbb{N}_0 \cup \{\infty\} \quad \text{where } \mathbb{N}_0 = \mathbb{N} \cup \{0\}.$$

- **Optimal cost-to-go (ground truth):**

$$h^*(s) = \min\{\text{cost}(p) \ : \ p \text{ is a path from } s \text{ to a goal}\}, \qquad \text{and } h^*(s) = \infty \text{ if no goal is reachable.}$$

- **Now let's define properties of heuristics:**
  - goal-aware
  - safe
  - admissible
  - consistent

Heuristic Formal Definitions

- **Goal-aware:**

$$\forall g \in G : \; h(g) = 0.$$

  - Meaning: at a goal, the estimated remaining cost is 0.

- **Safe (for pruning dead ends):**

$$h(s) = \infty \implies h^*(s) = \infty.$$

  - Meaning: if the heuristic says "dead end", then it truly cannot reach a goal.
  - So **pruning** based on $h(s) = \infty$ never removes a solvable branch.

Heuristic Formal Definitions

- **Admissible (never overestimates):**

$$\forall s \in S: \; h(s) \leq h^*(s).$$

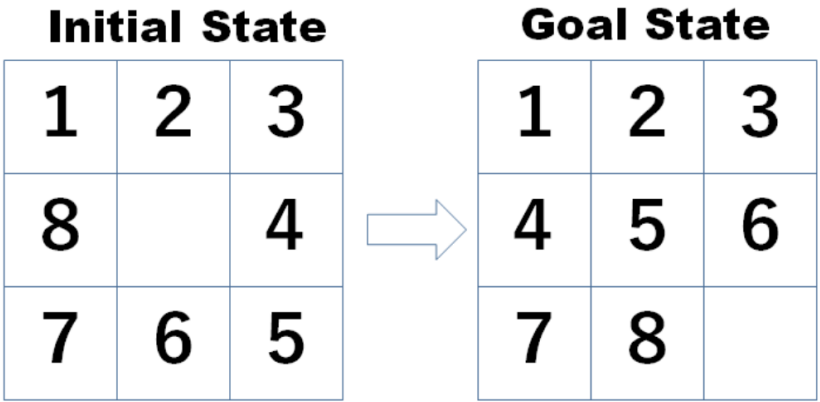
  - Intuition: can be optimistic, but not too high.

- **Consistent / Monotone (triangle inequality):**

$$\forall (u, v) \in E: \; h(u) \leq c(u, v) + h(v).$$

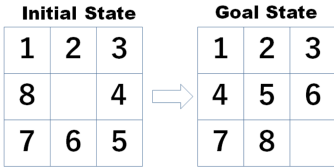
  - Intuition: estimates should improve smoothly along edges.

Example: 8-puzzle



Example: 8-puzzle

- **Board:** a 3×3 grid with tiles 1–8 and one blank □.
- **State s:** a placement (permutation) of the 8 tiles and the blank □ on the grid.
- **Start state *I*:** the given initial arrangement.
- **Goal set G:** the target arrangement.
- **Actions A:** move the blank one cell **up / down / left / right** (if within bounds).
- **Transition:** swap the blank with the adjacent tile in that direction.
- **Objective:** find a minimum-cost sequence of moves from s_0 to some $g \in G$.



8-Puzzle: Heuristics

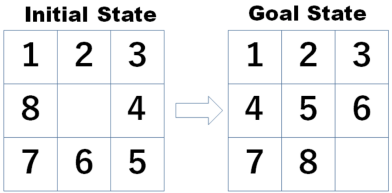
- **Let's see two common heuristics for the 8-puzzle:**
  - Misplaced tiles
  - Manhattan distance

8-Puzzle: Heuristics

- **Misplaced tiles (Hamming distance):**
  - For each numbered tile, count 1 if it is not in its goal position (ignore the blank).

$$h_{\text{mis}}(s) = \sum_{t \in \{1,\ldots,8\}} \mathbf{1}(\text{pos}_s(t) \neq \text{pos}_G(t))$$

$$h_{\text{mis}}(I) = 4$$



8-Puzzle: Heuristics

Is **misplaced tiles goal-aware** and **admissible**?

8-Puzzle: Heuristics

- **Goal-aware: yes.**
  - If s is a goal state, every tile is in place, so

$$h_{\text{mis}}(s) = 0.$$

- **Admissible: yes.**
  - Each move can place **at most one** tile into its correct position.
  - So you need at least one move per currently-misplaced tile, hence

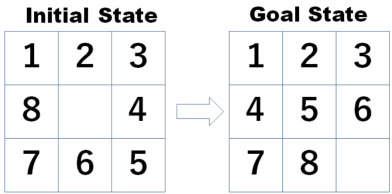
$$h_{\text{mis}}(s) \leq h^*(s).$$

8-Puzzle: Heuristics

- **Manhattan distance (tile-wise):**
  - For each tile, compute how many grid steps (up/down/left/right) it is away from its goal cell.
  - Sum over all numbered tiles (ignore the blank).

$$h_{\text{man}}(s) = \sum_{t \in \{1,\ldots,8\}} \Big(|x_s(t) - x_G(t)| + |y_s(t) - y_G(t)| \Big).$$

$$h_{\text{man}}(I) = 2 + 2 + 2 + 2 = 8$$



8-Puzzle: Heuristics

Is **Manhattan goal-aware** and **admissible**?

8-Puzzle: Heuristics

- **Goal-aware: yes.**
  - If s is a goal state, every tile is already at its goal cell, so every distance is 0:

$$h_{\text{man}}(s) = 0.$$

- **Admissible: yes.**
  - Consider any tile t. To reach its goal cell, it must change its row/column by

$$|x_s(t) - x_G(t)| + |y_s(t) - y_G(t)|.$$

  - The only way tile t changes position is when it is swapped with the blank, and each such swap moves it by **exactly one** grid step.
  - So tile t must be moved at least its Manhattan distance many times.
  - Therefore any solution needs at least: $\sum_{t=1}^{8} \Big(|x_s(t) - x_G(t)| + |y_s(t) - y_G(t)| \Big) = h_{\text{man}}(s)$ moves, hence $h_{\text{man}}(s) \leq h^*(s)$.

8-Puzzle: Heuristics

- **Definition:** A heuristic h_1 dominates a heuristic h_2 if

$$\forall s: \; h_1(s) \geq h_2(s)$$

and both heuristics are **admissible**.

- Manhattan dominates misplaced tiles.
  - For every state s:

$$h_{\text{man}}(s) \geq h_{\text{mis}}(s).$$

  - Reason: every misplaced tile has Manhattan distance at least 1, while correctly placed tiles contribute 0.
  - The dominating heuristic is usually more informative.

Outline

- ~~Part 1: Heuristics~~
- **Part 2: Informed Search Algorithms**
  1. **Systematic Search Algorithms**
     - **Best First Search**
       - ♢ GBFS (Greedy Best-First Search)
       - ♣ A* (A-star)
     - IDA* (Iterative Deepening A*)
  2. Local Search Algorithms

Best First Search

- Expand the "best" node first according to an **evaluation function** $f(n)$ and a **priority queue** (OPEN).
- "Best" means the node with the **smallest** $f(n)$ value at that moment.
- Different choices of f produce different informed search algorithms.

Best First Search

Skeleton (graph search):

1. Put the start node s in OPEN with key $f(s)$.
2. Repeat: pop the node n with minimum $f(n)$.
3. If n is a goal, return the path.
4. For each successor n': compute key $f(n')$ and insert/update in OPEN.
5. **Optional:** Maintain CLOSED to avoid re-expanding settled nodes

Best-First Search Algorithms

Change only the key f:

- **Uniform-Cost Search (UCS) / Dijkstra:** $f(n) = g(n)$.
- **Greedy Best-First Search (GBFS):** $f(n) = h(n)$.
- **A*:** $f(n) = g(n) + h(n)$.
- **Weighted A* (WA*):** $f(n) = g(n) + w \cdot h(n)$, $w \geq 1$.

Outline

- ~~Part 1: Heuristics~~
- **Part 2: Informed Search Algorithms**
  1. **Systematic Search Algorithms**
     - **Best First Search**
       - ♢ **GBFS (Greedy Best-First Search)**
       - ♣ A* (A-star)
     - IDA* (Iterative Deepening A*)
  2. Local Search Algorithms

GBFS (Greedy Best-First Search)

1. GBFS is one of the most commonly used algorithms for satisficing planning.
2. If the heuristic satisfies some very strong assumptions, GBFS can find a solution very quickly while exploring only a small part of the search space.

- **Idea:** always expand the node that **looks closest** to a goal according to the heuristic $h(n)$.
- **Priority rule:** OPEN is a priority queue keyed by $h(n)$
- **Key point:** GBFS ignores the path-cost so far $g(n)$ when choosing what to expand
  - It still stores $g(n)$ to report the final solution cost.

GBFS (Greedy Best-First Search)

- **Trade-offs:** GBFS can be very fast, it rushes toward the states that *look* closest to the goal. But it ignores the cost paid so far, so it is not guaranteed to be optimal: the path it returns may be much more expensive than the cheapest one.
- **Typical tie-breakers:** if several nodes share the same h, break ties by smaller g or by FIFO.
- **Key takeaway:** great for quick solutions when optimality is not required.

GBFS (Greedy Best-First Search)

GBFS Pseudocode

```
1  function GBFS(s_0):
2      OPEN   ← new min-priority-queue keyed by h(·)
3      CLOSED ← ∅
4      OPEN.push(make_root_node(s_0), key = h(s_0))
5      while not OPEN.empty():
6          σ ← OPEN.pop_min()      // smallest h
7          if is_goal(state(σ)):
8              return extract_solutionσ
9          if state(σ) ∉ CLOSED:
10             CLOSED.add(state(σ))
11             for each ⟨o, s⟩ in successors(state(σ)):
12                 σ' ← make_node(σ, o, s)
13                 if state(σ') ∉ CLOSED:
14                     OPEN.push(σ', key = h(s))
15     return unsolvable
```

Properties of Greedy Best-First Search

- **Completeness:**
  - **Yes** with the pseudocode shown in these slides.
  - In the lecture's version, GBFS is complete when using a **safe** heuristic.
- **Optimality:**
  - GBFS is **not** optimal.
  - Can return a suboptimal solution.

Let's run GBFS!

In the course GitHub, under **Rec-2**, you have the **Informed Search Lab**.

There you can run **GBFS** on one of the ready-made graphs, or on a graph you create.

Outline

- ~~Part 1: Heuristics~~
- **Part 2: Informed Search Algorithms**
  1. **Systematic Search Algorithms**
     - **Best First Search**
       - ♢ ~~GBFS (Greedy Best-First Search)~~
       - ♣ **A* (A-star)**
     - IDA* (Iterative Deepening A*)
  2. Local Search Algorithms

A*

1. A* is one of the most commonly used algorithms.
2. Often used for optimal planning.

- rank nodes by the evaluation function

$$f(n) = g(n) + h(n),$$

- where $g(n)$ is the cost from s_0 to n,
  and $h(n)$ estimates the cheapest remaining cost from n to a goal.
- Intuitively, each node n carries
  an estimate of the cost of a cheapest solution path that goes through n.

- **Uniform-Cost Search (UCS):** prioritize by $g(n)$ only.
- **Greedy Best-First Search (GBFS):** prioritize by $h(n)$ only.
- **A*:** prioritize by $f(n) = g(n) + h(n)$ (think: UCS + GBFS).

A* Pseudocode

A*

```
1  function Astar(s_0):
2      OPEN    ← new min-priority-queue keyed by f(σ) = g(σ) + h(σ)
3      CLOSED  ← ∅
4      DISTANCE ← ∅                          // distance to node
5      OPEN.push(make_root_node(s_0), key = [g(σ) = 0 + h(s_0)])
6      while not OPEN.empty():
7          σ ← OPEN.pop_min()                // node with smallest f
8          if state(σ) ∉ CLOSED or g(σ) < DISTANCE(state(σ)):
9              DISTANCE(state(σ)) ← g(σ)
10             CLOSED.add(state(σ))
11             if is_goal(state(σ)):
12                 return extract_solution(σ)
13             for each ⟨o, s⟩ in successors(state(σ)):
14                 σ' ← make_node(σ, o, s)
15                     OPEN.push(σ', key = g[σ'] + h(s))     // priority = f
16     return unsolvable
```

1. **Yes** with the pseudocode shown in these slides.
2. In the lecture's version, A* is complete when using a **safe** heuristic.

1. Assume all edge costs are non-negative.
2. with admissible h, A* returns an optimal (minimum-cost) solution (Proved in the lecture).

1. If h is consistent, then f is non-decreasing along any path.
2. Therefore (try to prove it!), when A* expands a state s, it expands it with the **minimal** g: $g(\sigma) = g^*(s)$, the cost of a cheapest path from s_0 to s.
3. So every state is expanded **at most once** in the whole search, the reopening check $g(\sigma) < \text{DISTANCE}(\text{state}(\sigma))$ never fires.

Notes:

1. Consistent + goal-aware $\Rightarrow$ Admissible
2. So A* returns an optimal solution.

Claim

If h is consistent and goal-aware, then h is admissible.

Proof.

Let s be any state, and let

$$s = s_0 \to s_1 \to \cdots \to s_k = g$$

be an optimal path from s to a goal state g, with total cost

$$h^*(s) = \sum_{i=0}^{k-1} c(s_i, s_{i+1}).$$

Since h is consistent,

$$\forall i : h(s_i) \leq c(s_i, s_{i+1}) + h(s_{i+1}).$$

Claim

If h is consistent and goal-aware, then h is admissible.

Proof.

Applying this repeatedly along the path gives

$$h(s) = h(s_0) \leq c(s_0, s_1) + h(s_1) \leq \cdots \leq \sum_{i=0}^{k-1} c(s_i, s_{i+1}) + h(g).$$

Because h is goal-aware, $h(g) = 0$. Therefore,

$$h(s) \leq \sum_{i=0}^{k-1} c(s_i, s_{i+1}) = h^*(s).$$

So for every state s,

$$h(s) \leq h^*(s),$$

and hence h is admissible. □

Let's run A*!

Back to the **Informed Search Lab**:
choose **A***, pick a graph (or build your own) and run it.

What Does a Heuristic Buy Us?

In Practice 1, UCS explored blindly in every direction. A* uses the heuristic to point the search toward the goal, and still returns an optimal solution. How much work does that save?

Solution length	4	8	12
UCS	112	6,300	3.6×10^6
A* with h_{mis}	13	39	227
A* with h_{man}	12	25	73

8-puzzle, search without a CLOSED list (states can repeat): average number of nodes expanded

Source: Klein & Abbeel, CS188 lecture slides, UC Berkeley (statistics by A. Moore)

What About Bigger Problems?

The 8-puzzle has only 181,440 reachable states. What happens when the state space explodes, or when the map is a real city?

15-puzzle (4×4, $\approx 10^{13}$ states)

Blind BFS, whole space	28 days, 1.4 TB
A* with h_{man}	$\approx$ 5 s per instance

Sources: Korf & Schultze, *Large-Scale Parallel Breadth-First Search*, AAAI 2005; Burns et al., *Implementing Fast Heuristic Search Code*, SoCS 2012

Tokyo road map

h = straight-line distance

(a road is never shorter than a straight line)

UCS	209,744 nodes
A*	28,254 nodes

Same route, about 7× fewer nodes

Source: Malacad, *Comparing Dijkstra's and A* Search Algorithm*, Medium, 2022

Same Maps, Five Algorithms

- Our best-first algorithms differ only in the key f.
- So let's run all of them on the same problems and see what each choice of f costs and saves.

Grid route finding with random walls, 8 move directions, h = straight-line distance. Totals over 6 maps.

Algorithm	Key $f(n)$	Nodes generated	Path cost
UCS	g	2,326,926	789
A*	$g + h$	175,217	789
WA*, $w = 1.4$	$g + 1.4 \cdot h$	49,373	797
WA*, $w = 2$	$g + 2 \cdot h$	46,569	835
GBFS	h	41,774	891

Source: Norvig, *aima-python*, `search4e` notebook (AIMA 4th ed.)

When Greed Pays Off

1. GBFS gives up the optimality guarantee.
2. Sometimes that trade is worth a lot: on hard instances A* may never finish, while GBFS solves them quickly.

	A*	GBFS
Heuristic 1	1,251,260	258
Heuristic 2	out of 8 GB on 3 instances	3,006

11-puzzle (3×4), average number of nodes expanded. Both algorithms use the same admissible heuristic in each row.

Source: Wilt & Ruml, *Effective Heuristics for Suboptimal Best-First Search*, JAIR 2016

Outline

- ~~Part 1: Heuristics~~
- **Part 2: Informed Search Algorithms**
  1. **Systematic Search Algorithms**
     - ~~Best First Search~~
     - **IDA* (Iterative Deepening A*)**
  2. Local Search Algorithms

IDA*

- **Purpose:** find an optimal (cheapest) path like A*, but using **tiny memory**.
  - With admissible h (and nonnegative costs), IDA* returns an optimal solution.
- **Key idea:** run a sequence of **depth-first searches**, each bounded by an f-cost bound.

$$f(n) = g(n) + h(n) \qquad \text{and we expand only nodes with} \qquad f(n) \leq \text{bound}.$$

- If no goal is found, increase the bound to the **smallest** f that exceeded it in the previous pass, and repeat.

Why IDA*?

- A* can run out of RAM (OPEN/CLOSED may grow exponentially).
- IDA* trades extra **time** (repeating work across iterations) for **linear memory** (stack only).

Algorithm (core loop)

- Start with a bound:
$$\text{bound} \leftarrow f(s_0) = g(s_0) + h(s_0) = h(s_0).$$
- Run a depth-first search (DFS) that **prunes** any node with $f(n) > \text{bound}$.
- Track the **minimum** $f(n)$ that exceeded the bound:
$$\text{next_bound} \leftarrow \min\{f(n) \ : \ f(n) > \text{bound}\}.$$
- If a goal is found under the current bound $\rightarrow$ return an optimal path.
- Otherwise set
$$\text{bound} \leftarrow \text{next_bound}$$
and repeat.

IDA* – Pseudocode

IDA* (main loop)

```
1  function IDAstar(s_0):
2      bound ← h(s_0)
3      path ← [ make_root_node(s_0) ]
4      while true:
5          res ← Search(path, bound)
6          if res = FOUND:
7              return extract_solution(path.last)
8          if res = ∞:
9              return unsolvable
10         bound ← res
```

IDA* – Pseudocode

IDA*

```
1  function Search(path, bound):
2      σ ← path.last
3      f ← g(σ) + h(state(σ))
4      if f > bound: return f
5      if is_goal(state(σ)): return FOUND
6      min ← ∞
7      for each ⟨o, s'⟩ in successors(state(σ)):
8          if s' not on path:
9              σ' ← make_node(σ, o, s')
10             path.push(σ')
11             t ← Search(path, bound)
12             if t = FOUND: return FOUND
13             if t < min: min ← t
14             path.pop()
15     return min
```

Let's run IDA*!

Back to the **Informed Search Lab**:
choose **IDA***, pick a graph (or build your own) and run it.

When IDA* Wins

IDA* repeats work in every iteration, so it expands many more nodes than A*. But it remembers almost nothing, so each node is cheap and memory never runs out.

	A*	IDA*
Total time	516 s	634 s
Memory	27 GB	12 MB
Nodes expanded	$1\times$	$\approx 12\times$
Nodes per second	3.0 million	29.1 million

15-puzzle, 100 random instances, h_{man}, a modern machine

Source: Burns et al., *Implementing Fast Heuristic Search Code*, SoCS 2012

In 1985 memory was tiny: A* ran out after storing about 30,000 nodes and solved 0 of the 100 instances, while IDA* solved all 100.

Source: Korf, *Depth-First Iterative-Deepening: An Optimal Admissible Tree Search*, AIJ 1985

When IDA* Struggles

Look back at the IDA* pseudocode. Where could it waste work?

When IDA* Struggles

Look back at the IDA* pseudocode. Where could it waste work?

- **Many different f values.** Each iteration raises the bound only to the next f value. If almost every node has its own f, each iteration adds about one new node and repeats everything before it.

  15-puzzle where moving tile t costs $\sqrt{t}$: IDA* solved no instance within one hour, and some need more than a week.

  Source: Burns & Ruml, *Iterative-Deepening Search with On-Line Tree Size Prediction*, AMAI 2013

When IDA* Struggles

Look back at the IDA* pseudocode. Where could it waste work?

- **Many different f values.** Each iteration raises the bound only to the next f value. If almost every node has its own f, each iteration adds about one new node and repeats everything before it.

  15-puzzle where moving tile t costs $\sqrt{t}$: IDA* solved no instance within one hour, and some need more than a week.

  Source: Burns & Ruml, *Iterative-Deepening Search with On-Line Tree Size Prediction*, AMAI 2013

- **No CLOSED list.** IDA* only avoids states on the current path. On a grid, many paths reach the same cell, and IDA* explores it again for each of them.

  Game maps: most plain IDA* runs did not finish. Even when given extra memory to remember visited states, it took 33.7 ms vs 1.7 ms for A*.

  Source: Björnsson et al., *Fringe Search: Beating A* at Pathfinding on Game Maps*, 2005

Outline

- ~~Part 1: Heuristics~~
- **Part 2: Informed Search Algorithms**
  1. ~~Systematic Search Algorithms~~
  2. **Local Search Algorithms**
     - **Beam Search**
     - EHC, if time permits and there is interest :)

Beam Search

- **Idea:** like best-first search, but we keep only a **limited frontier**.
- We maintain OPEN ordered by an evaluation $f(\sigma)$ (for example $h(\sigma)$ or $g(\sigma) + h(\sigma)$).
- **Beam width B:** after generating successors, keep only the **best** B nodes and discard the rest.
- **Why use it?**
  - Much smaller memory than A*/GBFS on large problems.
  - Often finds good solutions fast.
- **Trade-off:** not complete and not optimal in general (we may discard the path to the goal).

- The standard version in informed search is **Greedy Beam Search**:
  - $f(n) = h(n)$ (keep the B smallest heuristic values)
- But we can plug in other scores:
  - **A*-like beam:** $f(n) = g(n) + h(n)$
  - **Weighted beam:** $f(n) = g(n) + w \cdot h(n)$, where $w \geq 1$
- **Reminder:**
  pruning to a fixed B means Beam Search is generally **not complete** and **not optimal**.

Beam Search – Pseudocode

Beam Search

```
1  function BeamSearch(s_0, B ,f ):
2      if is_goal(s_0):
3          return extract_solution(make_root_node(s_0))
4      OPEN ← [ make_root_node(s_0) ]
5      CLOSED ← ∅
6      while OPEN not empty:
7          NEXT ← []
8          for each σ in OPEN:
9              if is_goal(state(σ)):
10                 return extract_solution(σ)
11             CLOSED.add(state(σ))
12             for each ⟨o, s⟩ in successors(state(σ)):
13                 σ' ← make_node(σ, o, s)
14                 if state(σ') ∉ CLOSED: NEXT.append(σ')
15         sort NEXT by f
16         OPEN ← first B nodes of NEXT    // prune to beam width
17     return unsolvable
```

Let's run Beam Search!

Back to the **Informed Search Lab**:
choose **Beam Search**, pick a graph (or build your own) and run it.

Choosing the Beam Width

The beam width B decides how much beam search remembers.

- With $B = 1$ it keeps only the best successor, just like hill climbing.
- With $B = \infty$ it keeps everything, just like BFS.

Let's see what happens in between on the 48-puzzle.

Choosing the Beam Width

The beam width B decides how much beam search remembers.

- With $B = 1$ it keeps only the best successor, just like hill climbing.
- With $B = \infty$ it keeps everything, just like BFS.

Let's see what happens in between on the 48-puzzle.

Beam width B	Solved	Avg. path length	Time
1	0%	–	–
5	100%	11,737	0.09 s
1,000	84%	1,338	2.8 s
10,000	80%	440	11.3 s
50,000	0% (out of memory)	–	–

48-puzzle (7×7), key $f = h_{man}$, 50 random instances, at most 6 million stored nodes

Source: Furcy & Koenig, *Limited Discrepancy Beam Search*, IJCAI 2005

Beam Search vs. WA*

How does beam search compare to a systematic algorithm on a large problem? Here is one FreeCell instance, run with WA* and with beam search.

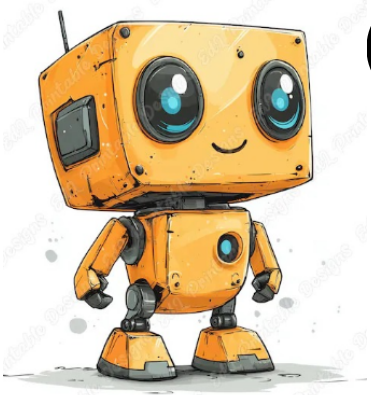
	Solved?	Nodes stored	Time
WA* ($w = 2$)	no (out of memory)	$>$ 5.6 million	$>$ 878 s
Beam search	yes, 27 moves	10,000	6.4 s

FreeCell, the solitaire card game, solved as a search problem

Source: Zhou & Hansen, *Breadth-First Heuristic Search*, KR 2004

Outline

- ~~Part 1: Heuristics~~
- **Part 2: Informed Search Algorithms**
  1. ~~Systematic Search Algorithms~~
  2. **Local Search Algorithms**
     - ~~Beam Search~~
     - **EHC, if time permits and there is interest :)**



Questions ?